GymApp

Revisión del proyecto frente al enunciado y la rúbrica

Reto 1 · 2º DAM · Aplicación de escritorio

Acceso a Datos (ADT) y Programación de Servicios y Procesos (PSP)

Revisión del 29/10/2026, después de montar el Cliente–Servidor, los backups con proceso secundario, el modo sin conexión, el Histórico y el borrado de cuenta (28/10). Sustituye a la revisión del 25/10. Entrega del Sprint 2: 04/11/2026.

Puntuación estimada con el código actual

ADT	PSP	TOTAL (ADT + PSP)
53,5 / 57	**50 / 50**	**103,5 / 107**
aprox. 94 %	aprox. 100 %	aprox. 97 %

*Estimación propia, punto por punto. Los puntos de "el alumno es capaz de explicar" se cuentan suponiendo una defensa correcta: **hay que saber explicar todo lo nuevo** (alumno elegido al azar). Antes: 25/10 por la mañana 31,5 · 25/10 por la tarde 46 · ahora 103,5. La nota real la pone el profesorado.*

Conclusión en una frase

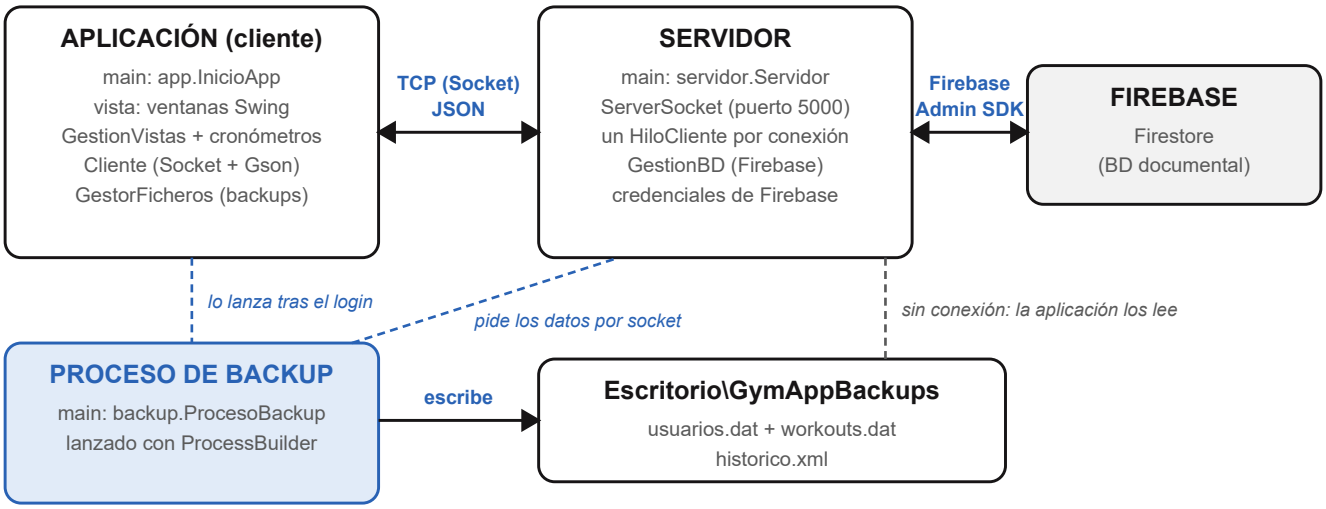
La arquitectura ya es la que pide el enunciado: **Cliente–Servidor con sockets TCP y JSON**, Firebase solo en el servidor, **backups .dat/.xml hechos por un proceso secundario** y **modo sin conexión**. Lo que queda en ADT es de organización del código (el acceso a datos en el modelo) y en PSP todo depende de saber explicarlo en la defensa.

Contenido

- 1. Cómo se ejecuta la aplicación (servidor primero, después la aplicación)
- 2. Qué se ha hecho el 28/10 (con el texto del enunciado y la rúbrica)
- 3. Rúbrica ADT punto por punto
- 4. Rúbrica PSP punto por punto
- 5. Lo que falta y riesgos
- 6. Dudas para preguntar al profesorado
- 7. Otros módulos del reto

1. Cómo se ejecuta la aplicación

Ahora son **dos programas** dentro del mismo proyecto, cada uno con su main. **Primero el servidor y después la aplicación.** Si solo se abre InicioApp, como antes, la aplicación no encuentra el servidor y entra en **modo sin conexión** (solo con un backup anterior o con admin/admin).



Pasos en Eclipse

#	Qué hacer	Qué se ve
1	Clic derecho en **src/servidor/Servidor.java** → Run As → Java Application.	Consola: “Servidor GymApp esperando clientes en el puerto 5000”. **Se deja abierto.** Cada petición que llega sale en esta consola (“Peticion recibida: LOGIN”…).
2	Clic derecho en **src/app/InicioApp.java** → Run As → Java Application.	La aplicación de siempre. Eclipse abre otra consola; con el icono de pantallas de la consola se cambia entre la del servidor y la de la aplicación.
3	Hacer login con un usuario.	En la consola de la aplicación salen las líneas “[Backup] …” del proceso secundario y “Proceso terminado con código 0”.
4	Para terminar: cerrar la aplicación y parar el servidor con el **cuadrado rojo** de su consola.	Si el servidor se queda abierto y se vuelve a arrancar, sale “Address already in use”: hay que parar el anterior.

Qué pasa al pulsar Entrar en el login

Situación	Qué hace la aplicación
admin / admin	Entra sin servidor y sin backup (modo sin conexión). Se mantiene por si el antivirus del instituto lo bloquea todo.
Servidor encendido	cliente.hayConexion() es true. El servidor comprueba el email y la contraseña en Firebase. Si son correctos se lanza el backup (proceso secundario) y se entra con datos del servidor; se puede guardar todo.
Servidor apagado	El login busca el usuario en usuarios.dat (backup) y comprueba la contraseña con BCrypt. Workouts de workouts.dat e histórico de historico.xml. Ajustes avisa “Modo sin conexión”. Los workouts que se hacen se guardan en pendientes_<id>.dat y se suben (con la subida de nivel) en el siguiente login con servidor. **Registrarse necesita el servidor.**
Servidor apagado y sin backup	Solo entra admin/admin (el backup se crea la primera vez que alguien entra con el servidor encendido).

Otros detalles

- **Varias aplicaciones a la vez:** se puede ejecutar InicioApp varias veces con un solo servidor; cada conexión la atiende su propio hilo (HiloCliente).

- **Servidor en otro ordenador:** en controlador/Cliente.java cambiar IP_SERVIDOR = "localhost" por la IP del ordenador del servidor (los dos en la misma red y el puerto 5000 abierto en el firewall).
- **Encender el servidor después:** si la aplicación ya estaba abierta, basta con volver a hacer login: cada petición abre su propia conexión y la comprobación se hace en cada login.
- **Datos iniciales:** InicioApp ya no sube los datos a Firebase en cada arranque. La llamada datosInicialesFB() está comentada en Servidor.main por si hay que volver a cargarlos.

2. Qué se ha hecho el 28/10

Todo probado con el servidor de verdad y Firebase (con dos usuarios de prueba que se crearon y se borraron después). Cada punto lleva el texto que lo pide.

Cliente–Servidor con sockets, hilos y JSON

> *"Una aplicación de escritorio (Cliente) se conecta a un Servidor, quien tiene el respaldo de una Base de Datos documental. La comunicación Cliente / Servidor se realiza mediante el protocolo TCP (sockets) y JSON."*
>
> **ENUNCIADO, «Arquitectura de la Aplicación», p. 7**

> *"Se utilizan Sockets TCP [...]. Existe el par Socket - Server Socket. [...] Se emplean Hilos para gestionar la comunicación mediante Sockets."*
>
> **RÚBRICA PSP, «Comunicación – Sockets»**

servidor/Servidor (ServerSocket, bucle accept) crea un **servidor/HiloCliente** (extends Thread) por conexión. **controlador/Cliente** tiene los mismos métodos que GestionBD y los pide por Socket: en GestionVistas solo cambió gestionBD.xxx() por cliente.xxx(). Una línea JSON de ida y otra de vuelta (Gson); **AdaptadorFecha** convierte los LocalDate. module-info: opens modelo to gson. 11 acciones.

Backups con proceso secundario y modo sin conexión

> *"Esta implementación se realiza en un proceso a parte para ser reutilizado en futuros proyectos. [...] Este proceso devuelve un mensaje con el resultado de su ejecución."*
>
> **ENUNCIADO, «Backups», p. 6**

> *"Este almacenamiento se realiza fuera de la estructura de proyecto. [...] No se incluyen los datos del Histórico."*
>
> **RÚBRICA ADT, «Ficheros Binarios»**

backup/ProcesoBackup es otro main que la aplicación lanza con **ProcessBuilder** después de cada login. Pide los datos al servidor y escribe usuarios.dat, workouts.dat (ObjectOutputStream, modelos Serializable, sin histórico) e historico.xml (DOM, mismo formato que el XML común) en C:\Users\<usuario>\Desktop\GymAppBackups. Escribe "OK: …" o "ERROR: …" y termina con código 0 o 1; la aplicación lo lee y espera con waitFor(). Sin servidor, el login y los datos salen de esos ficheros. La carpeta backups/ de dentro del proyecto se ha quitado. **29/10:** los workouts hechos sin conexión se guardan en pendientes_<id>.dat y se suben al servidor (y se sube de nivel) en el siguiente login con servidor.

Histórico, guardar el workout y subir de nivel

> *“En esta pantalla se muestran todos los workouts realizados por el usuario en orden cronológico inverso [...] El tiempo previsto se calcula dinámicamente [...]”*
>
> **ENUNCIADO, «Histórica workouts», p. 6**

> *“Al completar un workout de su nivel actual, se le desbloquea workouts del siguiente nivel.”*
>
> **ENUNCIADO, «Workouts», p. 3**

Nueva **VentanaHistorico** (desde la home y desde Workouts). Al terminar o salir de un workout se guarda en el histórico con su porcentaje (el aviso de SALIR ya lo dice: “Llevas el 66% del workout. Se guardará en tu histórico…”). Al completar al 100 % un workout de su nivel, sube de nivel y el resumen lo celebra.

Delete: eliminar la cuenta

> *“Se realiza al menos un tipo de cada operación CRUD en el Servidor.”*
>
> **RÚBRICA ADT, «Consultas»**

Botón rojo **ELIMINAR CUENTA** en Ajustes con aviso de confirmación. El servidor borra el histórico del usuario (Firestore no borra subcolecciones solo) y después el usuario. Después se rehace el backup.

Fallos arreglados de la revisión anterior

- **Id del usuario**: era “número de usuarios + 1”; con un borrado, el siguiente registro pisaba a otro usuario. Ahora es el id más alto + 1.
- **Registro**: si el email ya existía salía “Cuenta creada”. Ahora “Ese email ya está registrado”.
- **Login**: con un email que no existe decía “La contraseña no coincide”. Ahora “El email o la contraseña no son correctos”.
- **datosInicialesFB() en cada arranque**: quitado (comentado en el servidor).
- **agregarHistoricoPorUsuario** se saltaba el último histórico (size() - 1).
- **Usuario nuevo con “1 workout”**: el documento vacío del histórico se contaba. Ahora se salta.
- **Comentario “controlador falso”** de la cabecera de GestionVistas: cambiado.
- **obtenerListadoUsuarios** (del equipo) daba NullPointerException sin conexión.

3. Rúbrica ADT punto por punto

Columna **Ahora**: puntos que creemos que se sacarían hoy. **DEFENSA** = depende de lo que se explique en la presentación (alumno elegido al azar, la nota va al grupo).

Aspecto de la rúbrica	Máx	Estado	Ahora	Qué tenemos / qué falta
1. Login				
Login: credenciales contra el Servidor y opciones (acceso a Workouts / mensaje de error)	2	CUMPLE	**2**	**Hecho el 28/10:** el servidor comprueba email y contraseña en Firebase (acción LOGIN por socket). Error: “El email o la contraseña no son correctos”.
2. Workouts				
Listado de workouts del Servidor de su nivel e inferiores, con la info relevante **y la del cliente, especialmente su nivel**	3	CUMPLE	**3**	Los workouts llegan del servidor (OBTENER_WORKOUTS). Se ve el nombre y el nivel del usuario y la columna Vídeo.
Filtro por nivel concreto	2	CUMPLE	**2**	Desplegable TODOS / nivel.
Seleccionar un workout lleva a la pantalla de Ejercicios	1	CUMPLE	**1**	VER WORKOUT abre el detalle con la lista de ejercicios.
Volver: se puede regresar a la pantalla de Login	1	PARCIAL	**0,5**	ATRÁS vuelve a la home y desde ahí se cierra sesión. No es una vuelta directa al Login (ver dudas).
Se puede acceder a la pantalla de Históricos	1	CUMPLE	**1**	**Hecho el 28/10:** botón HISTÓRICO en Workouts (y HISTORIAL en la home).
3. Ejercicios				
Listado de los ejercicios del workout del servidor, con la info relevante **y la del cliente, especialmente su nivel**	3	CUMPLE	**3**	Detalle: ejercicios, series, descanso, duración y el nivel del usuario.
Información del ejercicio que el usuario va a hacer (el actual)	2	CUMPLE	**2**	Workout en curso: nombre, descripción, workout, series.
Información del **siguiente** ejercicio	2	CUMPLE	**2**	Detalle y Workout en curso muestran el siguiente ejercicio.
Se actualiza toda la info al completar el workout: tiempos, niveles del usuario, etc.	–	SIN PTS	**–**	**Hecho el 28/10:** se guarda en el histórico (tiempo y %) y sube de nivel al completar al 100 % un workout de su nivel.
Volver: se puede regresar a la pantalla de Workouts	1	CUMPLE	**1**	VOLVER en el Detalle y en el Resumen.
4. Histórico				
Listado del histórico del usuario guardado en el Servidor, ordenado (primero los recientes)	3	CUMPLE	**3**	**Hecho el 28/10:** VentanaHistorico. Datos del servidor (OBTENER_HISTORICO), ordenados por fecha, la más reciente arriba.
Cálculos automáticos del tiempo estimado y del % completado (no se almacenan, se muestran)	2	CUMPLE	**2**	El tiempo previsto se calcula al mostrarlo (series + descansos) y no se guarda. El % se calcula al terminar el workout; se guarda porque no hay otra forma de saberlo después (ver dudas).
Volver a Workouts	–	SIN PTS	**–**	VOLVER vuelve a Workouts (o a la home si se abrió desde allí).
5. Backups				

Aspecto de la rúbrica	Máx	Estado	Ahora	Qué tenemos / qué falta
Ficheros binarios .dat en cada arranque, fuera del proyecto, sobrescribiendo, con errores controlados, sin el histórico	3	CUMPLE	**3**	**Hecho el 28/10:** usuarios.dat y workouts.dat en C:\Users\<usuario>\Desktop\GymAppBackups, se sobrescriben, el histórico es transient. Se hacen tras cada login (ver dudas).
Modo offline: sin conexión con el Servidor, login con los datos del .dat	2	CUMPLE	**2**	Si hayConexion() es false, se busca el email en usuarios.dat y se comprueba la contraseña con BCrypt.
El alumno explica el proceso (.dat)	2	DEFENSA	**2**	DOCUMENTACION_BACKUPS.txt.
Ficheros .xml en cada arranque, fuera del proyecto, **solo** con el histórico	3	CUMPLE	**3**	historico.xml con la misma forma que el XML común del profesor (DOM + Transformer).
Modo offline: consultar el histórico desde el XML, con los cálculos	2	CUMPLE	**2**	Sin conexión el histórico se lee de historico.xml y la ventana hace los mismos cálculos.
El alumno explica el proceso (XML)	2	DEFENSA	**2**	DOCUMENTACION_BACKUPS.txt.
5. Comunicación				
El Cliente envía la información en JSON y Cliente y Servidor parsean los JSON	3	CUMPLE	**3**	**Hecho el 28/10:** una línea JSON por mensaje. Gson en los dos lados (toJsonTree / fromJson).
6. MVC				
La estructura (Cliente **y Servidor**) sigue MVC	3	PARCIAL	**1,5**	modelo / vista / controlador en el cliente, pero el acceso a datos (GestionBD, GestorFicheros) sigue en **controlador** y el servidor está en un paquete servidor/ propio.
El alumno explica y justifica la arquitectura	2	DEFENSA	**1**	Ahora hay servidor y es más fácil de justificar, pero queda lo del acceso a datos en el controlador.
7. Firebase				
Firebase bien configurado **en el Servidor**: credenciales, dependencias	2	PARCIAL	**1,5**	Solo el servidor usa Firebase y las credenciales. Pero es un solo proyecto: el fichero de credenciales está también en la parte del cliente (ver dudas).
Estructura de documentos correcta, **patrón DAO**, base de datos operativa	3	CUMPLE	**3**	Base de datos bien estructurada y operativa. **El profesor ha confirmado que no hace falta DAO**: valen las clases del modelo (POJOs: Usuario, Workout, Ejercicio, Serie, Historico) con GestionBD.
El alumno explica cómo se conecta a Firebase	2	DEFENSA	**2**	accesoFirebase().
Al menos una operación de cada tipo CRUD, errores típicos controlados (null, sin datos...)	3	CUMPLE	**3**	**Hecho el 28/10:** C (registro, guardar histórico), R (login, workouts, histórico), U (datos, contraseña, nivel) y **D (eliminar cuenta)**.
El alumno explica un CRUD o implementa uno sencillo al vuelo	2	DEFENSA	**2**	Depende de la defensa.

Total ADT estimado: 53,5 / 57

4. Rúbrica PSP punto por punto

Aspecto de la rúbrica	Máx	Estado	Ahora	Qué tenemos / qué falta
1. Backup (procesos)				
Ficheros .dat en cada arranque, mediante un **Proceso Secundario**	3	CUMPLE	**3**	**Hecho el 28/10:** backup.ProcesoBackup (otro main) lanzado con ProcessBuilder tras el login.
Offline: login con los datos del .dat si no hay conexión con el Servidor	2	CUMPLE	**2**	validarLoginSinConexion() con usuarios.dat.
Explicar el proceso **y hacer las modificaciones que se pidan**	2	DEFENSA	**2**	El proceso es una clase corta con un main: fácil de modificar.
Ficheros xml en cada arranque, mediante un Proceso Secundario	3	CUMPLE	**3**	El mismo proceso escribe historico.xml.
Offline: consultar los históricos desde el XML, con los cálculos	2	CUMPLE	**2**	obtenerHistoricoSinConexion() + VentanaHistorico.
Explicar el proceso y hacer las modificaciones que se pidan	2	DEFENSA	**2**	—
2. Cronos de Workout				
Crono workouts: empieza con el workout, se puede pausar y parar	3	CUMPLE	**3**	Hilo propio (Cronometro extends Thread).
Explicar el crono workouts	2	DEFENSA	**2**	DOCUMENTACION_VENTANA_WORKOUT_EN_CURSO.txt.
Crono ejercicios: empieza con el ejercicio, se para a la vez que el de workout	3	CUMPLE	**3**	Hilo propio, vuelve a 00:00 en cada ejercicio.
Explicar el crono ejercicios	2	DEFENSA	**2**	Igual que arriba.
Crono serie: **cuenta regresiva de 5 segundos**, después la estipulada, se pausa con el de workout, al acabar pasa al siguiente	3	CUMPLE	**3**	5 s de preparación y después la cuenta de la serie.
Explicar el crono serie	2	DEFENSA	**2**	Igual que arriba.
Crono descanso: empieza solo, cuenta regresiva, muestra el tiempo utilizado al empezar la siguiente serie	3	CUMPLE	**3**	“DESCANSO - USADO”.
Explicar el crono descanso	2	DEFENSA	**2**	Igual que arriba.
3. Comunicación				
Sockets TCP Cliente/Servidor, par Socket - ServerSocket, errores tratados	4	CUMPLE	**4**	**Hecho el 28/10:** ServerSocket(5000) en Servidor, Socket en Cliente. Servidor apagado, sin respuesta (timeout), puerto ocupado y petición mal hecha controlados.
Hilos para gestionar la comunicación por sockets (lo enseñado en clase)	4	CUMPLE	**4**	Un HiloCliente (extends Thread) por conexión. Confirmar que es el esquema visto en clase (ver dudas).
Explicar el proceso	2	DEFENSA	**2**	DOCUMENTACION_CLIENTE_SERVIDOR.txt.
JSON para enviar y recibir, parseo JSON a objetos y al revés, lógica coherente	4	CUMPLE	**4**	11 acciones, siempre con "ok". Usuario, Workout e Historico pasan de objeto a JSON y al revés.

Aspecto de la rúbrica	Máx	Estado	Ahora	Qué tenemos / qué falta
Explicar el proceso	2	DEFENSA	**2**	Igual que arriba.

Total PSP estimado: 50 / 50. Todo lo que puntúa está hecho; lo que queda es saber explicarlo y modificarlo en la defensa.

5. Lo que falta y riesgos

Lo que todavía resta puntos (ADT)

Qué falta	Puntos	Cómo se arreglaría
Acceso a datos en el modelo (MVC)	hasta +2,5	GestionBD y GestorFicheros son acceso a datos y el enunciado dice “Modelo: Incluirá todos los accesos a los datos”. Moverlos a modelo y explicar el MVC del servidor.
Firebase solo en el servidor	+0,5	Si el profesorado quiere dos proyectos, separar el paquete servidor/ (con GestionBD y las credenciales) en otro proyecto.
Volver a Login desde Workouts	+0,5	Preguntar si vale pasar por la home (ver dudas).

Requisitos del enunciado sin línea propia en la rúbrica

Requisito	Texto del enunciado	Estado
Foto de cada serie	“La información relevante a cada una de ellas es su nombre, número de repeticiones y foto explicativa.” (p. 5)	Falta: sale un cuadro gris.
Primer ejercicio no realizado	“Un botón nos permite seleccionar el primer ejercicio no realizado disponible del workout” (p. 4)	Siempre empieza en el ejercicio 1.

Riesgos

- **Saber explicarlo.** 25 de los 103,5 puntos son de “el alumno es capaz de explicar” (alumno al azar, la nota va al grupo). Todo el grupo tiene que leer DOCUMENTACION_CLIENTE_SERVIDOR.txt y DOCUMENTACION_BACKUPS.txt.
- **Un proyecto o dos.** Lo hemos hecho en un proyecto con dos main (decisión del grupo). La rúbrica habla de “La estructura del proyecto (Cliente y Servidor)”: confirmar con el profesorado.
- **Credenciales de Firebase** en assets/: no subirlas a un GitHub público (.gitignore o repositorio privado).
- **Workout de prueba “Prueba rápida”**: quitarlo antes de entregar (MOSTRAR_WORKOUT_PRUEBA = false). No se guarda en el histórico.

6. Dudas para preguntar al profesorado

- **¿Un proyecto con dos main o dos proyectos?** Tenemos Servidor e InicioApp en el mismo proyecto. Firebase solo lo usa el servidor.
- **¿Cuándo se hace el backup?** La rúbrica dice “Cada vez que se arranca el programa”; el enunciado dice “una vez el usuario se haya logueado”. Lo hacemos después de cada login con servidor.
- **Porcentaje completado**: la rúbrica dice que los cálculos “No se almacenan en ningun sitio”. El tiempo previsto no se guarda, pero el % sí (si no, después no se sabe cuántos ejercicios se hicieron). ¿Vale?
- **Hilos de los sockets**: un hilo nuevo por cada conexión, y cada petición abre y cierra su conexión. ¿Es el esquema visto en clase o prefieren una conexión abierta por cliente?
- **Volver desde Workouts**: la rúbrica dice “regresar a la pantalla de Login”. Tenemos Workouts → Home → Cerrar sesión. ¿Vale así?
- **Crono serie**: la cuenta de 5 segundos la hacemos antes de cada serie. ¿O solo antes de la primera?

7. Otros módulos del reto

- **Desarrollo de Interfaces:** aplicación web en Angular que lee gimnasio.json. Proyecto aparte.

- **SGE:** análisis en Excel / Google Sheets del XML del histórico y del JSON de workouts. Ya se genera historico.xml en GymAppBackups con el mismo formato que el XML común.
- **Sistemas Informáticos:** presupuestos, máquinas virtuales, carpetas compartidas, directivas y auditoría.
- **PMDM:** aplicación Android, con su propio documento de enunciado (no revisado aquí).